Data Quality-Recommender

A Deep Learning Data Quality Auditor for Text Datasets, with a Preprocessing Agent that Applies and Verifies the Fixes

Technical Report

Course: AASD 4014 Deep Learning I, Final Project

Group Members:

Rithika Bhargav Khandavilli (101669607)
Joao Marcos Gomes Da Silva (101477898)
Sergey Molkov (101563025)
Tharian James (101647371)

Project Manager: Rithika Bhargav Khandavilli

Date: October 6, 2026

Dataset: AG News Classification Dataset (Kaggle, amananandrai/ag-news-classification-dataset)

Framework: Python, PyTorch (models trained from scratch), Streamlit application

TABLE OF CONTENTS

		Page
1.0	**INTRODUCTION**	**5**
1.1	Problem statement	5
1.2	Data	6
1.2.1	Source and structure	6
1.2.2	Class balance and vocabulary	6
1.2.3	Correlation analysis	6
1.2.4	Hidden quality issues: a manual audit	7
1.3	Tasks	8
2.0	**METHODOLOGY**	**8**
2.1	Plan of attack	8
2.2	Preprocessing	9
2.3	BiGRU topic classifier	10
2.3.1	Architecture	10
2.3.2	Gated recurrence	11
2.3.3	Bidirectional reading and masked pooling	11
2.3.4	Training objective	12
2.4	Label auditor: confident learning with per-class thresholds	12
2.5	Denoising character autoencoder	13
2.5.1	Input features	13
2.5.2	Architecture	13
2.5.3	Objective	13
2.5.4	Scoring and length calibration	13
2.6	Benchmark model	14
2.7	The recommender	14
2.7.1	Issues and severity	14
2.7.2	Quality score	15
2.7.3	Ordered plan and outputs	15
2.8	The preprocessing agent	15
2.8.1	State	15
2.8.2	Action library	16
2.8.3	Algorithm	16
2.8.4	Decision rules	16
2.8.5	How the agent is evaluated	17
2.9	Benchmark design: injecting problems we can measure	17
2.10	Implementation and application	18
3.0	**RESULTS**	**18**
3.1	Hyperparameter tuning	18
3.2	Training, loss and overfitting	19
3.3	Evaluation on the test set	20
3.4	Example inputs and model predictions	22
3.5	Detection results on the benchmark	23
3.6	The preprocessing agent on the benchmark	24

3.7 Does the cleaning help a model? 25
3.8 Discussion 26
3.8.1 Questions the guideline asks us to answer 26
3.8.2 Did we have a benchmark model? 27
3.8.3 Did our plan of attack make sense? 27
3.9 Hardware, memory and run time 27

4.0 CONCLUSIONS 28

4.1 The problem we solved 28
4.2 Use cases 28
4.3 Limitations and future work 29
4.4 Lessons learned 29

5.0 REFERENCES 29

LIST OF FIGURES AND TABLES

Figures

Figure 1: Rows per class in the training and test sets 6
Figure 2: Most frequent words per class (descriptions, stop words removed) 6
Figure 3: Correlation of engineered text features and class indicators (training set) 7
Figure 4: Tokens per row after cleaning, with the IQR upper bound and the 99th percentile 10
Figure 5: Distribution of each class's self-confidence on the real AG News test set, with the per-class thresholds 12
Figure 6: Raw reconstruction error and calibrated z-score against the number of active n-grams (real test rows, shortened copies and generated gibberish) 14
Figure 7: Validation accuracy of the eight configurations (embedding / dropout / label smoothing) 19
Figure 8: BiGRU loss and accuracy per epoch on the training and validation sets 19
Figure 9: Autoencoder reconstruction loss (epoch 1, at 8.3e-4, is left out of the plot) 20
Figure 10: Confusion matrix on the test set 21
Figure 11: Recall of each detector on the injected issues 23
Figure 12: Benchmark quality score by dimension before and after applying the recommended plan 24
Figure 13: Quality score after each verified agent step on the benchmark 25
Figure 14: Accuracy of the same model trained on each version of the data 26
Figure 15: Change in accuracy after each agent step (one standard error is about 0.46 points) 26

Tables

Table 1: Hidden quality issues in the raw AG News training set 8
Table 2: Individual tasks: who did what 8
Table 3: Plan of attack and the evaluation chosen for each stage 9
Table 4: Preprocessing steps and their effect on the training set 9
Table 5: BiGRU architecture, tensor shapes and parameter counts 11
Table 6: Autoencoder architecture and parameter counts 13
Table 7: Severity rules of the recommender 15
Table 8: Quality score dimensions 15
Table 9: The agent's action library, in the order it is tried 16
Table 10: The 14 injected issue types (1,824 corrupted rows; duplicates add 266 rows, 7,866 in total) 17
Table 11: Code, data and artifacts 18
Table 12: Classifier results on the validation split and the untouched 7,600-row test set 20
Table 13: Per-class test metrics 21
Table 14: BiGRU predict() on our own sentences and on eight random AG News test rows (title shown) 22
Table 15: Autoencoder anomaly scores on example inputs (threshold z = 2.16) 22
Table 16: Detection quality of each detector on the benchmark 23
Table 17: Recall of the two junk checks: they complement each other 23
Table 18: Top suspected mislabels in the real, unmodified test set (261 flagged, 3.4%) 24
Table 19: Agent log on the benchmark (start score 86.2, 7,866 rows) 25
Table 20: Hardware, memory and run time 27
Table 21: Use cases of DQ-Recommender 29

Introduction

Most of the time in a machine learning project goes into finding and fixing problems in the data: missing values, duplicates, broken encodings, very sparse features, wrong labels and junk rows. These problems are usually found by hand, and often only after a model behaves strangely. This report describes **DQ-Recommender**, a system that audits a labelled text dataset automatically and writes a data quality report that says what is wrong, how bad it is, and which preprocessing steps to apply in which order. A **preprocessing agent** then applies those steps one at a time, verifies each one, and proves the result by retraining a model on the cleaned data.

The system has four parts: a statistical profiler; two deep learning detectors trained from scratch in PyTorch on the AG News dataset [1], [2] (a bidirectional GRU topic classifier used as a label auditor, and a denoising autoencoder on character n-grams that flags garbled or off-domain text); a recommender that turns the findings into severity-ranked issues, a 0 to 100 quality score and an ordered preprocessing plan; and the preprocessing agent. Everything is packaged in a Streamlit application. The project follows course option 1 (train and evaluate RNN models on text) with an option 2 element (an agent that solves a real data-cleaning task).

1.1 Problem statement

Think of a news aggregator, or any company that trains models on text it collects, that receives a new batch of labelled articles every week from scrapers, partners and annotators. Before a batch can be used for training somebody has to check it, and in practice the batches contain:

- missing titles or descriptions, and blank strings or 'N/A' placeholders that `df.isna()` does not see;
- duplicated stories (the same wire story from several outlets), which inflate validation scores when copies land in both train and test;
- broken HTML entities and markup left over from scraping;
- very sparse bag-of-words representations and a long tail of rare words, which decide the representation and vocabulary size;
- wrong labels, and junk rows such as gibberish, boilerplate, code or product listings.

The last two cannot be caught with rules. A model that has learned what a World or Sports story looks like can point at rows whose label disagrees with the text, and a model that has learned what normal news text looks like can point at rows that are not news at all. That is where deep learning helps.

Objective. Given any CSV of labelled text, automatically detect the data quality issues, quantify them, recommend an ordered preprocessing plan, and let an agent apply and verify the fixes.

Success criteria.

- The classifier must be accurate enough to be trusted as a label auditor (target: over 90% test accuracy).
- On a benchmark with issues we inject ourselves, so that the ground truth is known, the system must find most of them (recall) without many false alarms (precision, ROC-AUC).
- The cleaning must make a model trained on the data better, not only raise a quality score.

1.2 Data

1.2.1 Source and structure

AG News (Zhang, Zhao and LeCun, 2015) [1] is built from AG's corpus of more than one million news articles gathered from over 2,000 sources. The Kaggle version [2] has 120,000 training rows and 7,600 test rows with three columns: Class Index (1 World, 2 Sports, 3 Business, 4 Sci/Tech), Title and Description. The average row has 6.8 words in the title and 31.1 in the description.

1.2.2 Class balance and vocabulary

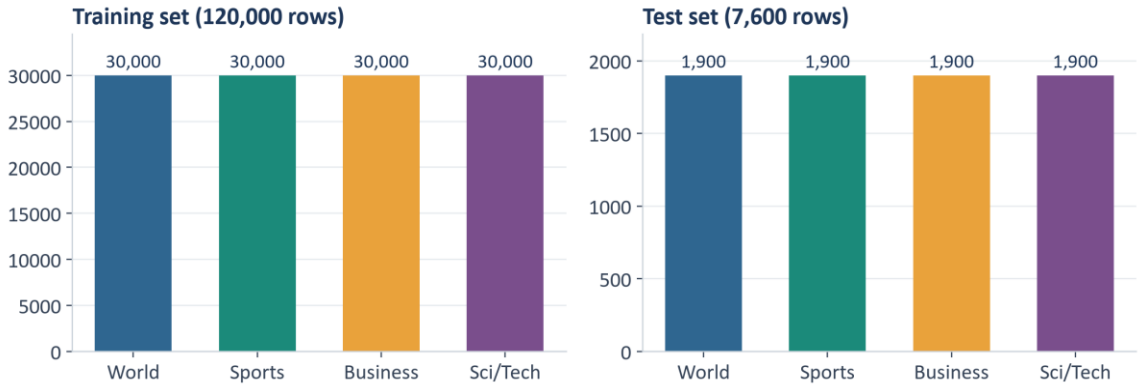


Figure 1: Rows per class in the training and test sets

The classes are perfectly balanced (30,000 and 1,900 rows per class), so no resampling or class weights were needed for training. Because uploaded datasets are often imbalanced, the recommender still measures balance (imbalance ratio and normalised entropy) and recommends class weights or stratified splits when it finds a skew.

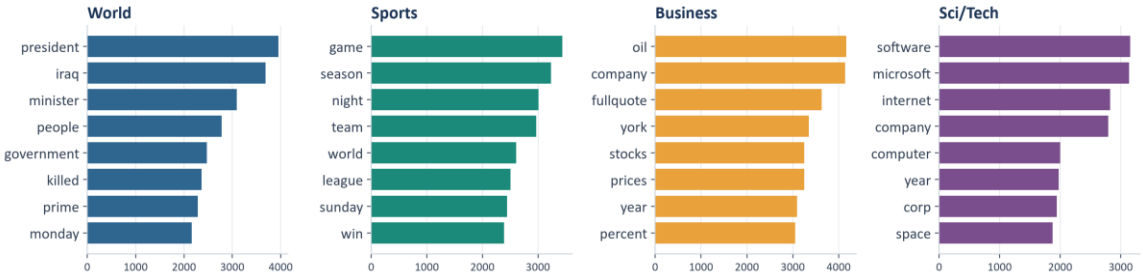


Figure 2: Most frequent words per class (descriptions, stop words removed)

The topics are strongly keyword driven (*iraq*, *president* for World; *game*, *season* for Sports; *oil*, *company* for Business; *software*, *microsoft* for Sci/Tech). This already suggests that a linear model on word counts will be a strong baseline.

1.2.3 Correlation analysis

AG News has no numeric columns, so we built a numeric view of each row (lengths, digit, upper-case and punctuation ratios, a news-source prefix flag and the number of broken entities) and correlated it with one-hot class indicators. A Pearson correlation with a 0/1 indicator is the point-biserial correlation.

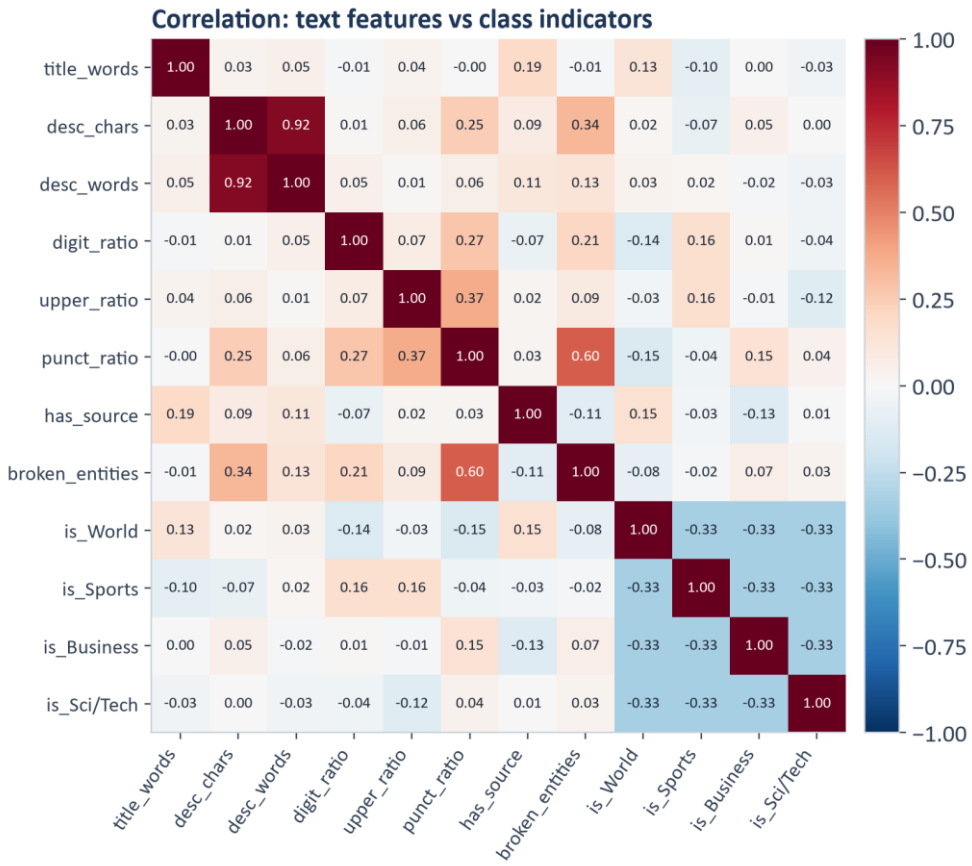


Figure 3: Correlation of engineered text features and class indicators (training set)

- **Redundancy.** Description length in characters and in words are almost the same variable (r = 0.92), so only one should be kept as a feature.
- **Weak class links.** The class indicators correlate only weakly with any surface feature (|r| at most 0.16). Digits and capitals lean towards Sports (scores, team names) and a source prefix leans towards World.
- **A quality signal.** `broken_entities` and `punct_ratio` correlate at r = 0.60: HTML debris inflates punctuation, which makes it a usable data quality signal.

The conclusion that informed the model design: the topic is not decided by length or formatting, so we need a model of the words. That motivated a recurrent network over learned word embeddings.

1.2.4 Hidden quality issues: a manual audit

`df.isna()` reports no missing values and the classes are balanced, so the data looks clean. It is not (Table 1).

Table 1: Hidden quality issues in the raw AG News training set

Finding	Rows	Share
Broken HTML entities such as #39; or quot;	29,132	24.3%
Literal backslashes where line breaks used to be	12,522	10.4%
Escaped tags such as <b>	2,675	2.2%
Duplicate descriptions	1,277	1.1%
Rows whose normalised text appears with more than one label	56	0.05%
Test rows whose text also appears in the training set (leakage)	15	0.2% of test
Rows with an Excel error string ('#NAME?')	1	

About 16% of descriptions start with a news-source prefix (Reuters, AP, AFP), and some sources are almost exclusive to one class (Canadian Press appears only in World). A model can partly learn who wrote the story instead of what it is about, a mild form of shortcut learning. We kept the prefix because it is real information at prediction time.

1.3 Tasks

The project was split into four work packages plus project management. Table 2 lists who did what; every member presents the part of the system they built.

Table 2: Individual tasks: who did what

Member	Student ID	Tasks and deliverables
Tharian James	101647371	Exploratory data analysis (class balance, length distributions, top words, correlation matrix); manual data quality audit; preprocessing pipeline (text repair, duplicates, conflicting labels, leakage, outliers, vocabulary, sparsity); system overview.
Joao Marcos Gomes Da Silva	101477898	BiGRU classifier architecture and PyTorch implementation; hyperparameter grid search; training with early stopping and TensorBoard; overfitting analysis; test evaluation and TF-IDF benchmark; example predictions; confident-learning label auditor.
Sergey Molkov	101563025	Character autoencoder (features, architecture, training, length calibration); statistical profiler and recommender (severity, quality score, ordered plan); benchmark with 14 injected issue types and detection evaluation.
Rithika Bhargav Khandavilli	101669607	Preprocessing agent (observe, plan, act, verify loop) and its downstream evaluation; reflection and conclusions; Streamlit application and live demo; integration of the parts.
Project Manager: Rithika Bhargav Khandavilli		Planning, final review, presentation, report, problem statement.

2.0 METHODOLOGY

2.1 Plan of attack

We split the work into six stages and agreed up front how each stage would be evaluated (Table 3).

Table 3: Plan of attack and the evaluation chosen for each stage

Stage	What we did	How it is evaluated
1. EDA and manual audit	Class balance, lengths, top words, correlation matrix, hidden quality issues	Findings drive the preprocessing decisions
2. Preprocessing	Text repair, duplicates, conflicts, leakage, outlier handling, vocabulary	Coverage, OOV rate, rows kept
3. Models	BiGRU classifier and denoising character autoencoder, both from scratch	Loss curves, validation accuracy
4. Tuning and evaluation	2 x 2 x 2 grid, early stopping, test set, TF-IDF baseline	Accuracy, macro-F1, confusion matrix, calibration
5. Recommender	Profiler + both models turned into issues, severities, a 0 to 100 score and a plan	Recall, precision, ROC-AUC on 14 injected issue types
6. Agent and app	Observe, plan, act, verify loop; Streamlit demo	Retrain a model on the data before and after

The key design decision was to evaluate the detectors, not only the classifier. We built a benchmark by injecting known problems into the test set, which neither model saw during training, so the recall and precision of every check can be measured against ground truth (Section 2.9).

2.2 Preprocessing

All preprocessing lives in `dq/text_utils.py` so that the notebook and the application apply exactly the same steps (Table 4).

Table 4: Preprocessing steps and their effect on the training set

Step	Effect on the training set	Reason
Repair encodings: fix bare entities, unescape twice, strip tags and URLs, NFKC, collapse spaces	29,132 rows repaired	Junk tokens such as '#39;' pollute the vocabulary
Drop groups of identical text with different labels	105 rows	The true label cannot be known
Drop duplicates, keep the first copy	264 rows	Copies inflate validation scores
Remove training rows that also appear in the test set	32 rows	Leakage
Drop rows with fewer than 5 tokens	1 row	No signal
Truncate sequences at 71 tokens (99th percentile)	0.96% of rows truncated	Long rows are real stories
Vocabulary of 30,000 words, minimum frequency 2	98.7% token coverage, 1.35% OOV	Coverage curve flattens after 30k
Stratified 90/10 train/validation split	107,638 / 11,960 rows	Same class mix in both

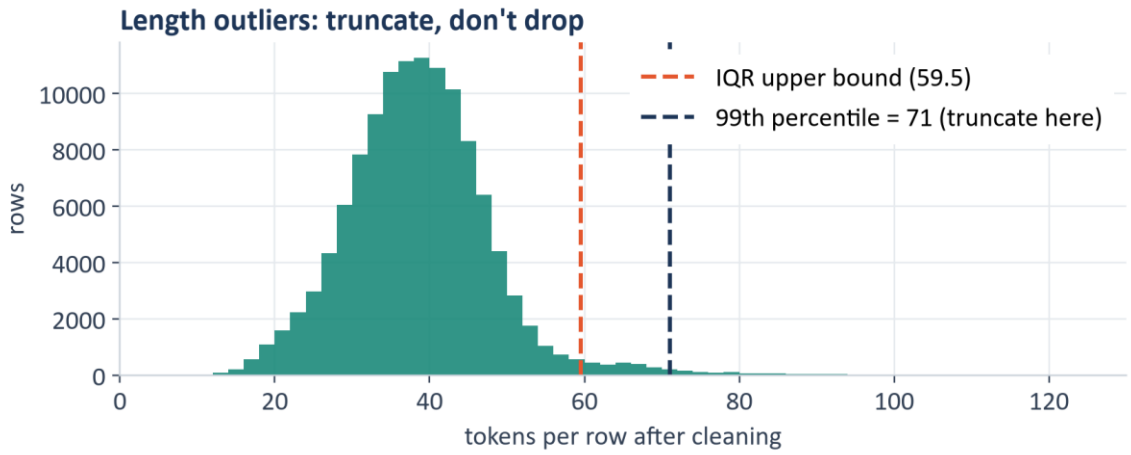


Figure 4: Tokens per row after cleaning, with the IQR upper bound and the 99th percentile

Outliers. The IQR rule marks 3,320 rows above 59.5 tokens as outliers. They are not errors, only longer news stories, so dropping them would throw away real data. Instead we truncate at the 99th percentile (71 tokens), which affects 0.96% of rows, and drop only the near-empty rows.

Imbalance. Class imbalance is absent here (Section 1.2.2), so no resampling was needed.

Sparsity. The bag-of-words matrix is 99.95% zeros: each story uses about 32 distinct words out of 67,507, so a dense float32 copy would need 32.3 GB while the sparse CSR matrix needs 46 MB. That is why the classifier uses a learned dense embedding rather than one-hot or count vectors.

2.3 BiGRU topic classifier

The system uses two neural networks trained from scratch in PyTorch [10] and an agent that acts on their output. The classifier recognises the topic of a text and can therefore spot rows whose label disagrees with the text.

2.3.1 Architecture

Each row is cleaned, tokenised into lower-case words, mapped to vocabulary ids (0 is padding, 1 is unknown) and padded or truncated to T = 71 tokens. For a batch of B rows the tensor shapes through the network are shown in Table 5.

Table 5: BiGRU architecture, tensor shapes and parameter counts

Layer	Output shape	Parameters	Notes
Token ids	B × 71		30,000-word vocabulary
Embedding(30,000, 200), padding_idx 0	B × 71 × 200	6,000,000	Dropout 0.15 on the embeddings
Bidirectional GRU, H = 128 per direction	B × 71 × 256	253,440	2 × 3H(E + H + 2)
Masked max pooling ⊕ masked mean pooling	B × 512	0	Padding positions excluded
BatchNorm1d(512), Dropout 0.3	B × 512	1,024	Normalises the pooled features
Linear 512 → 128, ReLU	B × 128	65,664	
BatchNorm1d(128), Dropout 0.3	B × 128	256	
Linear 128 → 4	B × 4	516	Logits; softmax gives p(class)
Total		**6,320,900**	**95% of it in the embedding table**

2.3.2 Gated recurrence

At each time step t the GRU [3] combines the embedded token x_t with the previous hidden state h_{t-1} through two gates. The update gate z_t decides how much of the old state to keep and the reset gate r_t decides how much of it feeds the new candidate state:

$$z_t = \sigma(W_z\, x_t + U_z\, h_{t-1} + b_z)$$
$$r_t = \sigma(W_r\, x_t + U_r\, h_{t-1} + b_r)$$
$$\tilde{h}_t = \tanh(W_h\, x_t + U_h\, (r_t \odot h_{t-1}) + b_h)$$
$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

σ is the logistic sigmoid and $\odot$ the element-wise product. Because the new state is an interpolation between the old state and the candidate, gradients can flow through $(1 - z_t)$ over many steps, which avoids most of the vanishing gradient problem of a plain RNN. A GRU has three weight blocks where an LSTM has four, so it has 25% fewer recurrent weights; on 30 to 70-token news snippets it reaches the same accuracy and trains faster. We also clip the gradient norm at 1.0, which keeps the recurrent updates stable early in training.

2.3.3 Bidirectional reading and masked pooling

One GRU reads the sequence left to right and a second reads it right to left [4]. Their states are concatenated, so every position sees both contexts ("shares fell" and "fell to rebels" need the word after "fell" as well as before). With the padding mask $m_t = 1[x_t \neq \text{pad}]$:

$$H_t = [\, h\rightarrow_t \,;\, h\leftarrow_t \,] \in \mathbb{R}^{256}$$
$$\max = \max_t \left(H_t - 10^4\,(1 - m_t) \right), \quad \text{mean} = \Sigma_t\, m_t\, H_t \,/\, \Sigma_t\, m_t$$
$$v = [\, \max \,;\, \text{mean} \,] \in \mathbb{R}^{512}$$

Max pooling picks the strongest topic cue anywhere in the text (for example "touchdown" or "Nasdaq"), mean pooling summarises the whole text, and subtracting 10^4 at padded positions guarantees that padding can never win the maximum. Taking only the last hidden state would make short and long texts behave differently, which pooling avoids.

2.3.4 Training objective

The head maps v to four logits. Training minimises cross-entropy against label-smoothed targets [5] with $\varepsilon = 0.1$ and $K = 4$ classes, so the true class gets 0.925 and every other class 0.025:

$$q_k = (1 - \varepsilon)\, 1[k = y] + \varepsilon / K$$

$$L = -\Sigma_k\, q_k \log \mathrm{softmax}(W v + b)_k$$

Label smoothing stops the network from pushing probabilities to exactly 0 or 1, which keeps them calibrated [9]; that matters because the label auditor reads the probabilities directly. Optimisation uses AdamW [6] (learning rate 2e-3, decoupled weight decay 1e-4), batch size 256, ReduceLROnPlateau (halve the learning rate after one epoch without validation improvement) and early stopping with patience 3, keeping the weights of the best validation-loss epoch. Validation loss is always plain cross-entropy so that runs with and without smoothing are comparable.

2.4 Label auditor: confident learning with per-class thresholds

A classifier with calibrated probabilities can audit labels. Following confident learning [7], for every class j the threshold t_j is the average probability the model gives to class j on the rows labelled j. A row x with given label y is flagged when the model is at least as confident about some other class k as it normally is on genuine rows of k:

$$t_j = \mathrm{mean}\,\{\, p_j(x) : \mathrm{label}(x) = j \,\}$$

$$\mathrm{flag}(x) \;\Leftrightarrow\; \exists\, k \neq y :\; p_k(x) \geq t_k$$

$$\mathrm{suggested}(x) = \mathrm{argmax}_{k \neq y}\, p_k(x) \;\text{ over the k with } p_k(x) \geq t_k, \qquad \mathrm{margin} = p_{\mathrm{suggested}} - p_y$$

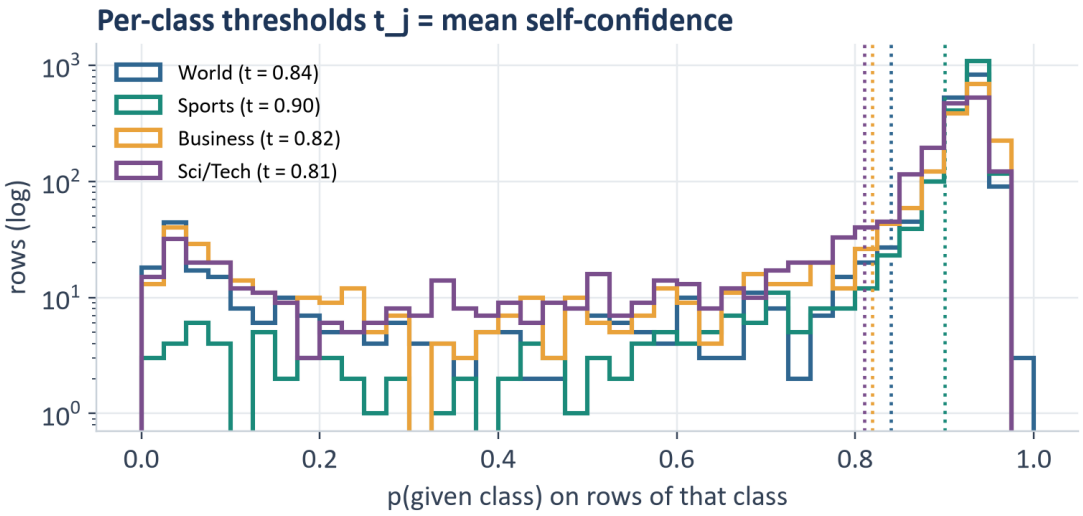


Figure 5: Distribution of each class's self-confidence on the real AG News test set, with the per-class thresholds

On the real test set the thresholds are 0.84 (World), 0.90 (Sports), 0.82 (Business) and 0.81 (Sci/Tech). Per-class thresholds matter because the classes differ: a single global cut-off would either miss Sports errors or over-flag the naturally less certain Business and Sci/Tech rows. Flagged rows are ranked by the margin, so a reviewer sees the clearest cases first. Rows with fewer than 12 tokens (for example a title with no description) are not judged, because there is

too little text to decide a topic; the completeness checks catch them instead. On the unmodified test set the auditor flags 261 rows (3.4%).

2.5 Denoising character autoencoder

The autoencoder learns what normal news text looks like and can therefore spot rows that are not news at all.

2.5.1 Input features

Each cleaned text is turned into character 2 to 4-grams within word boundaries (analyzer `char_wb`), hashed into d = 4,096 buckets with scikit-learn's HashingVectorizer [11] and L2 normalised. Hashing is stateless, so there is no vocabulary to fit or ship and the application rebuilds exactly the same features. Character features were chosen over words for a specific reason: gibberish made of unknown words maps every token to <unk>, which gives a word-level autoencoder an almost constant input and no signal, while character n-grams always exist and random or misspelled text produces n-grams that normal news rarely contains.

Worked example. The headline 'Oil hits record high' becomes 51 n-grams: ' o', 'oi', 'il', 'l ', ' oi', 'oil', 'il ', ' oil', 'oil ', then the same pattern for 'hits', 'record' and 'high'. They fall into 47 distinct buckets, which after L2 normalisation each carry a weight of about 0.15. A full-length news row activates about 388 buckets.

2.5.2 Architecture

Table 6: Autoencoder architecture and parameter counts

Block	Layers	Output	Parameters
Input corruption	Dropout(0.1) on the input vector (training only)	4,096	0
Encoder	Linear 4096 → 512, BatchNorm, ReLU, Dropout 0.2	512	2,098,688
Bottleneck	Linear 512 → 64, BatchNorm, ReLU	64	32,960
Decoder	Linear 64 → 512, ReLU	512	33,280
Reconstruction	Linear 512 → 4096 (no activation)	4,096	2,101,248
Total			**4,266,176**

2.5.3 Objective

With f the encoder, g the decoder and $\tilde{x}$ the corrupted input, training minimises the mean squared reconstruction error of the clean input [8] with Adam (learning rate 1e-3, batch 512, 12 epochs):

$$\tilde{x} = \text{dropout}(x, 0.1), \quad L = (1/d) \parallel g(f(\tilde{x})) - x \parallel^2$$

Because part of the input is removed, the network cannot learn the identity map; it has to learn the co-occurrence structure of English news n-grams and compress it into 64 numbers. Text with a different structure (random characters, code, product listings, log lines) reconstructs badly. The model is trained only on the cleaned training texts, so it learns what clean news looks like. The sparse matrices are densified one batch at a time, which keeps GPU memory at 0.28 GB and avoids a dense 107,638 × 4,096 matrix.

2.5.4 Scoring and length calibration

The raw anomaly score is the per-row reconstruction error $e = (1/d) \|\hat{x} - x\|^2$. A first version used one threshold on e, and testing showed it flagged short but normal texts: a short text has

few active n-grams, each carries a large weight after L2 normalisation, and its error is naturally higher (Figure 6, left). We therefore calibrate the score on clean validation rows plus shortened copies of them (3 to 11 words). Rows are split into 20 bins b by the number of active n-grams, and inside each bin the log error becomes a robust z-score using the median and the median absolute deviation (MAD):

$$z = (\log e - \text{median}_b) / (1.4826 \cdot \text{MAD}_b)$$

$$\text{flag} \Leftrightarrow z > \tau, \quad \tau = 99\text{th percentile of } z \text{ on clean text} = 2.16$$

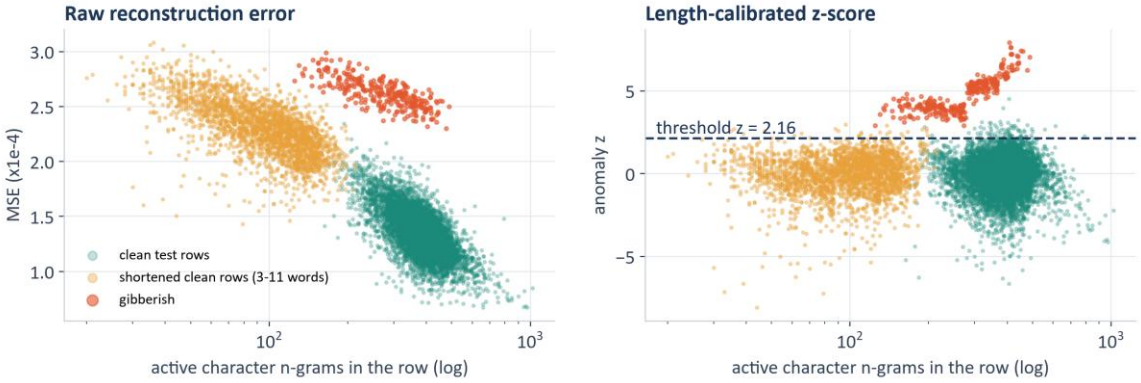


Figure 6: Raw reconstruction error and calibrated z-score against the number of active n-grams (real test rows, shortened copies and generated gibberish)

The factor 1.4826 makes the MAD a consistent estimate of the standard deviation for normal data, and the median and MAD are not pulled by the anomalies themselves. On the real test set a single raw threshold flags 95.7% of shortened clean rows; after calibration only 1.1% are flagged, the same rate as full-length rows (1.1%), while all generated gibberish is still flagged. A vocabulary rule complements the autoencoder: rows where more than 30% of the tokens are out of vocabulary are flagged as garbled, which is the better detector for heavy misspellings made of real characters.

2.6 Benchmark model

TF-IDF on unigrams and bigrams (minimum document frequency 2, sublinear term frequency, up to 200,000 features) with logistic regression (C = 4), trained on the same split. It tells us whether the deep model is worth it. The same model family is used to measure the agent's downstream effect (Section 3.7).

2.7 The recommender

The recommender turns evidence into an actionable report. It has three evidence sources: a statistical profiler (missing values, blank strings and placeholders such as 'N/A', exact, normalised and near duplicates with TF-IDF cosine of at least 0.9, conflicting labels, bag-of-words sparsity and vocabulary coverage, encoding artifacts, length and numeric outliers, class balance and correlations), the BiGRU label auditor (Section 2.4) and the autoencoder with the out-of-vocabulary rule (Section 2.5).

2.7.1 Issues and severity

Every finding becomes an Issue object with a category, a title, the number and share of affected rows, an explanation, a recommendation, a code snippet and its step in the preprocessing plan. Severity follows from the share of rows affected (Table 7).

Table 7: Severity rules of the recommender

Severity	Share of rows affected	Example on the benchmark
critical	≥ 20%	none on the benchmark
high	≥ 5%	encoding artifacts in Description (26.9%), suspected mislabels (9.7%)
medium	≥ 1%	missing titles (3.4%), exact duplicates (1.9%), near duplicates (1.4%)
low	< 1%	invalid labels (0.4%), ALL-CAPS titles (0.7%)
info	advice only	vocabulary size, sequence length

2.7.2 Quality score

Five dimension scores, each clipped to 0 to 100, combine into one weighted score with a letter grade (A from 90, B from 80, C from 70, D from 60). The weights are renormalised when the data has no label column (Table 8).

Table 8: Quality score dimensions

Dimension	Score	Weight
Completeness	100 − % missing cells (NaN, blank, placeholder)	0.30
Uniqueness	100 − (% duplicate rows + % near-duplicate rows)	0.20
Validity	100 − (½ % artifact rows + % short, junk, OOV or empty rows + ½ Σ % outliers)	0.30
Label quality	100 − 2 × % rows with missing, invalid, conflicting or suspected wrong labels	0.25
Balance	100 × normalised entropy of the class distribution	0.10

$$\text{overall} = \Sigma_d \, w_d \, s_d \, / \, \Sigma_d \, w_d$$

2.7.3 Ordered plan and outputs

Issues are sorted into a fixed pipeline so that each step works on data the previous steps have already made consistent: 0 standardise disguised missing values, 1 drop constant and identifier columns, 2 drop empty texts and invalid labels, 3 repair text, 4 remove duplicates and resolve conflicts, 5 handle near duplicates (grouped split), 6 remove short, junk and OOV-dominated rows, 7 choose the vocabulary, 8 set the sequence length and clip outliers, 9 scale and transform numeric columns, 10 handle class imbalance, 11 review or fix label noise. The report is written as HTML and Markdown, with an issues CSV and a ready-to-run `preprocess.py` containing the code of every step.

2.8 The preprocessing agent

The recommender writes a report; the agent (`dq/agent.py`) acts on it. It is a deterministic, tool-using agent: its tools are preprocessing actions, its observations are full data quality audits, and every decision it makes is checked against a new audit before it is kept.

2.8.1 State

After every action the agent re-runs the audit and stores an assessment with four parts: the statistical profile (missing and disguised-missing values, duplicates, near duplicates, label conflicts, sparsity, outliers, balance), the list of issues with the number of affected rows, the overall quality score and its five dimension scores. The model evidence (BiGRU label-noise table and autoencoder anomaly table) is computed once and indexed by the original row ids, so it stays valid as rows are removed.

2.8.2 Action library

The agent has 15 actions. Each declares the issue types it targets, whether it is destructive (removes rows) and a plain-language reason that is shown to the user. They are tried in the order of Table 9, which puts cheap, safe repairs before row removal and label changes last, when the data the classifier judges is already clean.

Table 9: The agent's action library, in the order it is tried

#	Action	Targets	Kind
1	Standardise disguised missing values ('N/A', blanks → NaN)	disguised_missing	repair
2	Drop constant and identifier columns	constant_, id_	repair
3	Drop rows with no text at all	empty_text	remove
4	Drop rows with missing or invalid labels	invalid_labels	remove
5	Fill a missing text field from the other one (first n words)	missing_<text col>	repair
6	Impute numeric (median + indicator) / categorical ('Unknown')	missing_<other col>	repair
7	Repair encodings and markup	artifacts_	repair
8	Resolve conflicting labels by majority vote, drop ties	label_conflict	remove
9	Remove exact and normalised duplicates	dup_rows, dup_text	remove
10	Remove near duplicates (TF-IDF cosine ≥ 0.9)	near_dup	remove
11	Remove junk rows (autoencoder flag or OOV > 30%)	anomalies, high_oov	remove
12	Remove truncated texts (< 3 words)	very_short	remove
13	Fix suspected mislabels (policy below)	label_noise	remove
14	Clip numeric outliers to the 1st to 99th percentile	outliers_	repair
15	Log-transform skewed numeric columns	skew_	repair

2.8.3 Algorithm

```
state = audit(df);  n0 = len(df)                        # observe
for action in LIBRARY:                                  # plan: fixed priority order
    if state.affected(action.targets) == 0: continue    #   skip issues that are not present
    new = action.apply(copy(df), state)                 # act on a copy
    if new is None: log('nothing to change'); continue
    if action.destructive and n0 - len(new) > 0.20 * n0:
        log('refused: row-loss budget'); continue       # budget guard
    after = audit(new)                                  # verify
    if after.score + 0.05 < state.score:
        log('rolled back')                              # never accept a worse dataset
    else:
        df, state = new, after                          # commit
        log(observation, detail, why, score/target before and after)
advise(state)   # vocabulary size, sequence length, class weights, grouped split
```

2.8.4 Decision rules

- **Row-loss budget.** A destructive action is refused if the total rows removed since the start would exceed 20% of the original data. This stops a chain of individually reasonable deletions from destroying the dataset.
- **Verify and roll back.** A change is kept only if the re-audited score is not lower than before (tolerance 0.05 points). Otherwise the copy is discarded and the step is logged as rolled back.

- **Mislabel policy.** `relabel_confident` (default) relabels a flagged row to the classifier's suggestion when p(suggested) ≥ 0.90 and drops the rest; `relabel_all`, `drop_all` and `flag_only` are the alternatives. Handled rows are unflagged so they are not reported again.
- **Conflicts.** Groups of identical text with different labels take the majority label; groups without a strict majority are dropped.
- **Advice.** After the loop the agent adds non-destructive advice computed from the final profile: the vocabulary size needed for 95% token coverage, a sequence length at the 95th percentile of row length, class weights when the largest-to-smallest class ratio exceeds 1.5, and a grouped train/test split when near duplicates remain.

2.8.5 How the agent is evaluated

A higher quality score is only a proxy. We therefore train the same TF-IDF + logistic regression model on the data before and after the agent and score both on 5,000 clean AG News training rows whose text does not appear in the benchmark. When the training data was repaired, the evaluation texts get the same repair, as a deployed pipeline would. To see which steps matter, the model is retrained after every applied step (cumulative). With n = 5,000 evaluation rows and accuracy near 88%, one standard error is:

$$\mathrm{SE} = \sqrt{(\,0.88 \cdot 0.12 \,/\, 5000\,)} \approx 0.46 \text{ points}$$

which separates real effects from noise.

2.9 Benchmark design: injecting problems we can measure

Real data has no answer key, so we made one. Each corruption type was injected into its own rows of the 7,600-row AG News test set, which neither model saw during training (random seed 7), and the affected row ids were saved as ground truth (Table 10).

Table 10: The 14 injected issue types (1,824 corrupted rows; duplicates add 266 rows, 7,866 in total)

Injected issue	Rows	Simulates	Should be caught by
Label flip (5%)	380	annotation mistakes	BiGRU label auditor
Missing title / 'N/A' title	228 / 38	broken scraper fields	Profiler: missing, placeholder
Missing / blank description	152 / 76	empty fields	Profiler: missing, blank
Gibberish	152	junk rows	Autoencoder + OOV rule
Heavy typos	152	OCR noise	Autoencoder + OOV rule
Off-domain (code, logs, product listings)	76	wrong source	Autoencoder + OOV rule
HTML markup + URLs	152	unstripped web pages	Profiler: artifacts
Truncated description (2 words)	114	cut-off records	Profiler: very short
Invalid label (0, 5, 'unknown', NaN)	38	bad label values	Profiler: label domain
Exact / near / conflicting duplicates	152 / 76 / 38	repeated stories	Profiler: duplicates, conflicts

The full audit (profiler, BiGRU and autoencoder) runs on the 7,866 rows in 13.8 s. Each detector is then compared with the truth: recall is the share of injected rows it flags, precision the share of its flags that are injected rows, ROC-AUC measures how well its score ranks injected rows above clean ones, and the false-alarm rate is measured on the 6,042 untouched rows.

2.10 Implementation and application

The notebook and the application share one Python package, so every number in this report can be reproduced from the same code (Table 11).

Table 11: Code, data and artifacts

Path	Contents
DQ_Recommender_AG_News.ipynb	Executed notebook with all code, outputs and figures
dq/	Shared package: text_utils, models, profiler, dl_checks, recommender, cleaning, agent
app.py, run_app.bat	Streamlit application [12]
artifacts/	Trained weights (classifier.pt, autoencoder.pt), vocabulary, calibration (config.json)
data/	train.csv and test.csv from Kaggle
sample_data/	Benchmark with injected issues and its ground truth, reference rows, example reports
runs/	TensorBoard logs (tensorboard --logdir runs)

The application has six sections: **Overview** (score, grade and dimension scores), **Issues** (severity-ranked findings with explanations and code), **Preprocessing agent** (run the agent and read its step-by-step log), **Data profile** (columns, missing values, lengths, balance, correlations), **Deep learning checks** (suspected mislabels and anomalies with their scores) and **Report & export** (HTML and Markdown report, issues CSV, `preprocess.py`, cleaned data). The models are loaded once and cached, and the whole application runs on a CPU.

How to run. Install the requirements (`pip install -r requirements.txt`), then run `streamlit run app.py`; it opens at http://localhost:8501. In the sidebar pick the benchmark or upload a CSV, choose the text and label columns, press *Run data quality audit*, then open *Preprocessing agent* and run it. To reproduce everything, run all cells of the notebook; it downloads the data with kagglehub or reads `data/train.csv` and `data/test.csv`.

3.0 RESULTS

3.1 Hyperparameter tuning

We tuned embedding size (128 or 200), dropout (0.3 or 0.5) and label smoothing (0 or 0.1): eight configurations, each trained for 3 epochs on a 60,000-row subset and scored on the full validation set with plain cross-entropy so the runs are comparable. The learning rate (2e-3) and GRU size (128) were fixed after earlier trial runs. The search took 183 s.

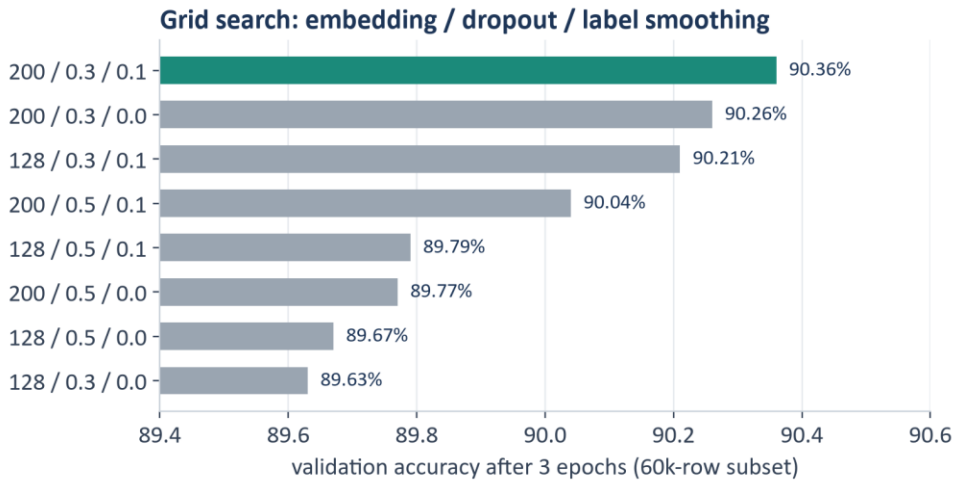


Figure 7: Validation accuracy of the eight configurations (embedding / dropout / label smoothing)

- **Embedding size.** The 200-dimensional embedding beat 128 in all four pairs: more capacity for the long tail of rare words.
- **Label smoothing** helped in three of four pairs. Softer targets also give better-calibrated probabilities, which matters because the auditor reads them.
- **Dropout.** Dropout 0.3 beat 0.5 in three of four pairs: at this short training length 0.5 underfits.
- **Sensitivity.** The whole grid spans only 0.7 points (89.63% to 90.36%), so the architecture matters more than these settings.

Selected configuration: embedding 200, dropout 0.3, label smoothing 0.1, hidden size 128, learning rate 2e-3.

3.2 Training, loss and overfitting

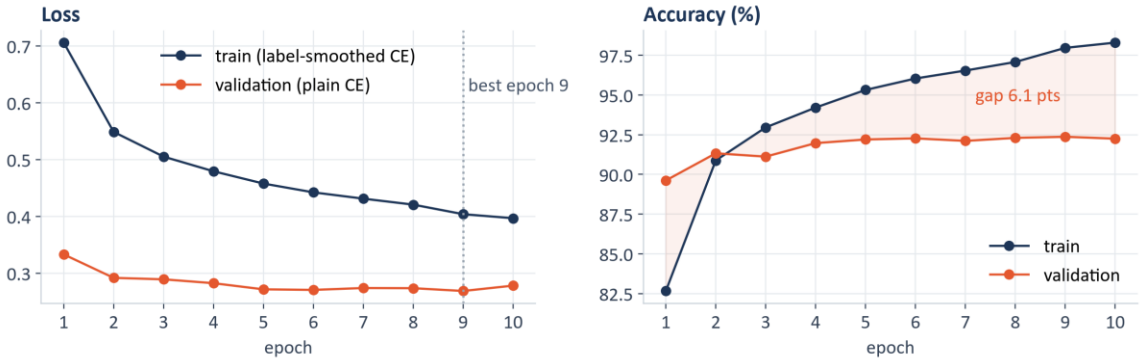


Figure 8: BiGRU loss and accuracy per epoch on the training and validation sets

The final model trained for 10 epochs in 142 s on the GPU (peak memory 1.3 GB). Losses and accuracies were also written to TensorBoard (`runs/classifier`). Validation accuracy rises to 92.2% by epoch 5 and then stays between 92.1% and 92.4%. The best validation loss is at epoch 9 (0.2686, accuracy 92.37%), which early stopping keeps. The learning rate was halved at epoch 8.

Is the classifier overfitted? Yes, mildly. Training accuracy keeps rising to 98.3% while validation accuracy flattens at about 92.3%, a gap of 6.1 points: after epoch 5 the network memorises training rows rather than learning patterns that generalise. The overfit is contained: validation loss does not climb sharply (0.2705 at epoch 6, 0.2780 at epoch 10) thanks to dropout, BatchNorm, label smoothing, weight decay and the learning-rate reduction, and early stopping keeps the best epoch. The training loss sits above the validation loss because the training loss is label-smoothed and computed with dropout active, while the validation loss is plain cross-entropy without dropout.



Figure 9: Autoencoder reconstruction loss (epoch 1, at 8.3e-4, is left out of the plot)

The autoencoder does not overfit: training and validation loss fall together for all 12 epochs (47 s, peak GPU memory 0.28 GB), and the validation loss is slightly below the training loss because input dropout is only active in training.

3.3 Evaluation on the test set

Table 12: Classifier results on the validation split and the untouched 7,600-row test set

Model	Validation accuracy	Test accuracy	Test macro-F1
BiGRU (ours)	92.37% (best epoch)	91.97%	0.920
TF-IDF + logistic regression (benchmark)	92.56%	92.32%	not computed

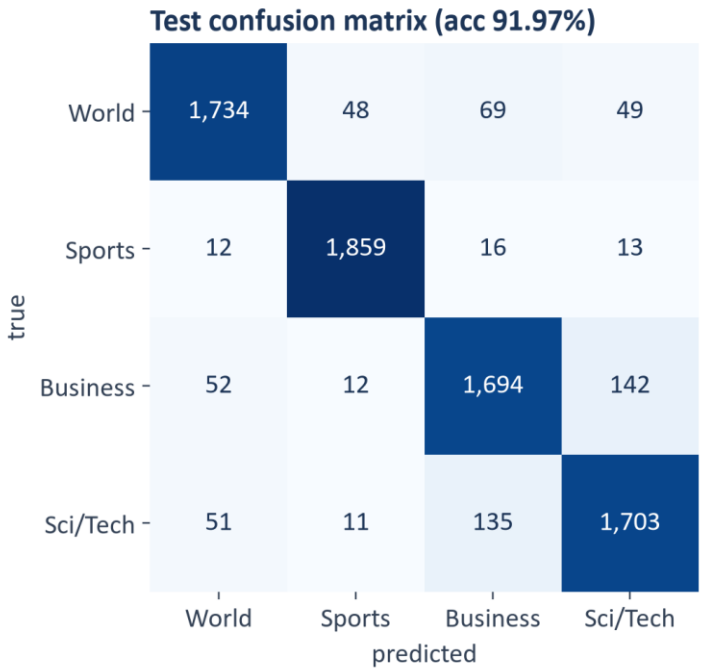


Figure 10: Confusion matrix on the test set

Table 13: Per-class test metrics

Class	Precision	Recall	F1	Most common error
World	0.938	0.913	0.925	predicted Business (69)
Sports	0.963	0.978	0.971	very few errors
Business	0.885	0.892	0.888	predicted Sci/Tech (142)
Sci/Tech	0.893	0.896	0.895	predicted Business (135)

Sports is the easiest class. Business and Sci/Tech are the hardest and are mostly confused with each other, which makes sense: stories about Apple, Google or the Sprint and Nextel merger are genuinely both. The model is well calibrated (expected calibration error 0.040 [9]), which is what the label auditor needs.

Benchmark comparison. We report it as it is: the TF-IDF benchmark is 0.35 points better than our from-scratch BiGRU on test accuracy. On short news snippets with strong keyword cues (Section 1.2.2), a well-regularised linear model on sparse n-grams is a very strong baseline, and a recurrent model without pretrained embeddings does not beat it. In this project the classifier is a means rather than the end: what matters is that it gives calibrated probabilities for label auditing, and the detection results below show it does.

3.4 Example inputs and model predictions

Table 14: BiGRU predict() on our own sentences and on eight random AG News test rows (title shown)

Input text	Predicted	Prob.	True label
Stocks rallied on Wall Street after the Federal Reserve left interest rates unchanged.	Business	0.92	(own example)
The Lakers beat the Celtics 112-104 behind a late fourth-quarter run.	Sports	0.68	(own example)
Researchers unveiled a new chip that doubles battery life in smartphones.	Sci/Tech	0.84	(own example)
Peace talks resumed in Geneva as the UN pressed both sides for a ceasefire.	World	0.97	(own example)
Apple shares fell after the company reported weaker iPhone sales.	Business (Sci/Tech 0.34)	0.50	(own example)
Retailers Vie for Back-To-School Buyers (Reuters)	Business	0.86	Business
Motorola buy adds heat to mesh networking	Sci/Tech	0.88	Sci/Tech
Steel group sees continued strength in China demand	Business	0.96	Business
Iraq's Neighbors to Get Little for Environment Loss (Reuters)	Business	0.40	Sci/Tech
Hughes allowed to speak to Rovers	Sports	0.93	Sports
D.C. to Face K.C. for MLS Championship (AP)	Sports	0.94	Sports
Yanks beat Pedro again; Santana wins 20th	Sports	0.90	Sports
UN Signs Pact with New World Court Opposed by U.S.	World	0.85	World

The ambiguous Apple headline is split between Business and Sci/Tech, the same confusion as in the matrix. The one wrong test prediction ('Iraq's Neighbors to Get Little for Environment Loss', labelled Sci/Tech) has low confidence (0.40), which is how the auditor separates uncertain rows from confident disagreements.

Table 15: Autoencoder anomaly scores on example inputs (threshold z = 2.16)

Input	Reconstruction error	Anomaly z	Flagged
Real news: 'Oil prices climbed to a record as traders worried about supply from Iraq.'	1.84e-4	-1.66	no
Short headline: 'Oil hits record high'	2.69e-4	0.65	no
Heavy typos of the real sentence	2.05e-4	-0.24	no (OOV rule catches it)
Random characters	2.99e-4	3.65	yes
Python code	2.67e-4	2.59	yes
Product listing (SKU, qty, price)	2.75e-4	2.98	yes

Note the short headline: without length calibration its raw error (2.69e-4) is as high as the code example, and it would have been flagged.

3.5 Detection results on the benchmark

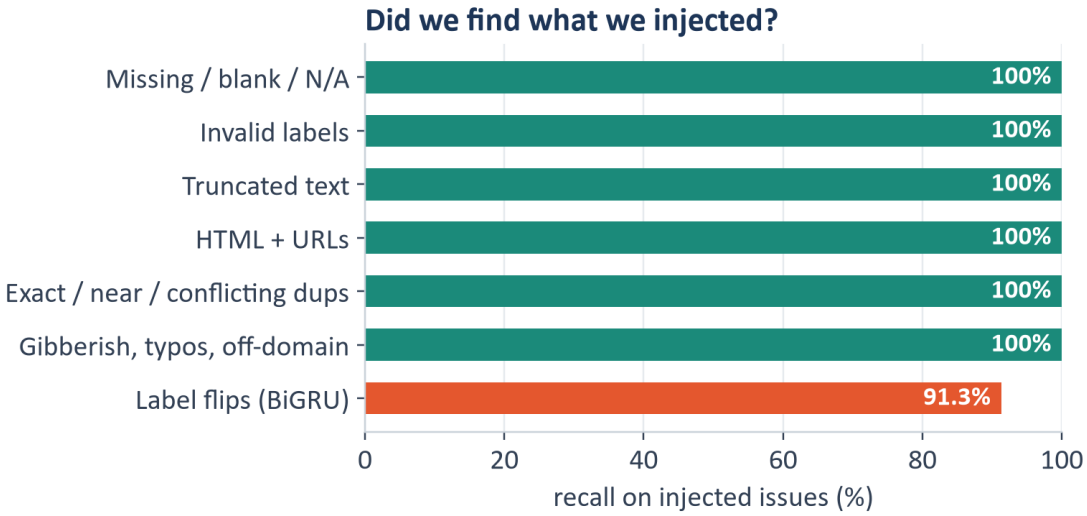


Figure 11: Recall of each detector on the injected issues

Table 16: Detection quality of each detector on the benchmark

Detector	Rows flagged	Recall	Precision	ROC-AUC
Rule checks (10 issue types)		100% for every type	exact rules	
BiGRU label auditor	762	91.3% of flips (347 / 380)	50% flips, 66% any injected issue	0.975
Autoencoder alone	381	79% of junk rows	79% junk, 82% any injected issue	0.983
Autoencoder + OOV > 30% rule		100% of junk rows		

Table 17: Recall of the two junk checks: they complement each other

Junk type	Autoencoder	OOV > 30%	Either
Gibberish	100%	100%	100%
Heavy typos	57%	100%	100%
Off-domain	82%	68%	100%

The autoencoder catches text made of real words or symbols that simply is not news (code, product listings, log lines), the OOV rule catches heavy misspellings, and together they find every junk row with 1.1% false alarms on untouched rows. Only half of the label auditor's 762 flags are our flips, but two thirds carry some injected issue and most of the rest are genuine ambiguity or label noise that was already in AG News (Table 18).

Table 18: Top suspected mislabels in the real, unmodified test set (261 flagged, 3.4%)

Real AG News test row (unmodified)	Given	Suggested	p(given)	p(suggested)
Toyota: Some security firms promise too much	Sci/Tech	Business	0.007	0.979
Nepal blockade 'blow to tourism'	Business	World	0.010	0.975
Boston archbishop reveals anguish of closings	Sports	Business	0.015	0.969
Special to ESPN.com: what do you give to the man...	Business	Sports	0.012	0.965
Nextel, Sprint Talk Merger	Sci/Tech	Business	0.022	0.957

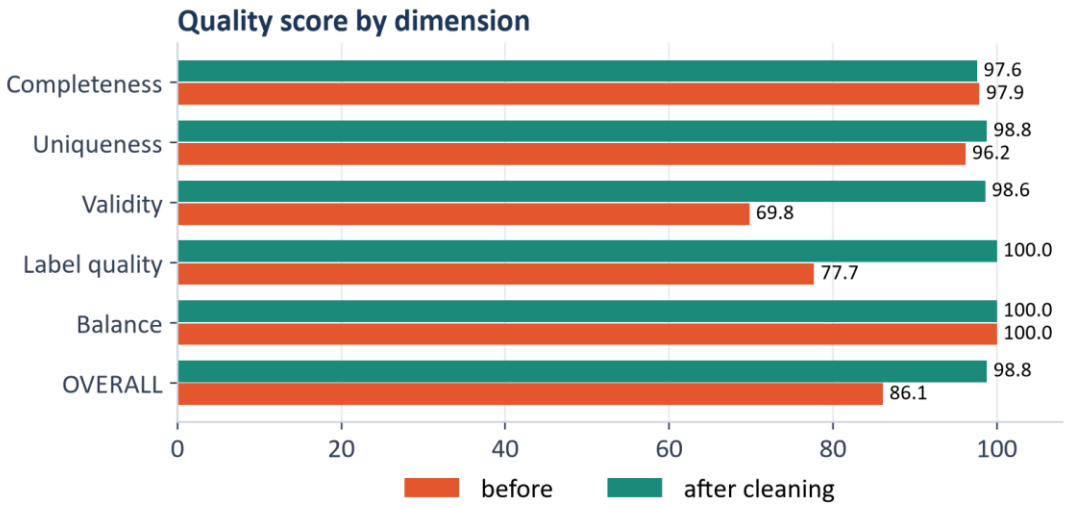


Figure 12: Benchmark quality score by dimension before and after applying the recommended plan

Applying the recommended plan raised the benchmark score from 86.1 (grade B) to 98.8 (A): validity rose from 69.8 to 98.6 and label quality from 77.7 to 100. The drop-based cleaner removed all gibberish, typo, off-domain, truncated, invalid-label and exact-duplicate rows and 91% of label flips, at a cost of 330 of 6,042 untouched rows (5.5%). The original, unmodified test set scores 93.0 (A).

3.6 The preprocessing agent on the benchmark

Section 2.8 describes the agent. Here it runs on the benchmark with its defaults: a 20% row-loss budget, the `relabel_confident` mislabel policy with a 0.90 confidence threshold, and the same BiGRU and autoencoder evidence as the recommender (Table 19).

Table 19: Agent log on the benchmark (start score 86.2, 7,866 rows)

Step	Status	Rows removed	Cells changed	Score after
Standardise disguised missing values	applied	0	114	86.2
Drop rows with missing or invalid labels	applied	38	0	86.4
Fill missing text from the other field	applied	0	494	86.9
Repair encoding and markup	applied	0	4,568	91.6
Resolve conflicting labels	applied	74	3	92.2
Remove exact and normalised duplicates	applied	155	0	92.4
Remove near duplicates	applied	22	0	92.5
Remove junk rows	applied	466	0	95.3
Remove truncated texts	applied	114	0	95.7
Fix suspected mislabels	applied	212	412	99.6
Vocabulary size / sequence length; grouped split	advice	0	0	99.6

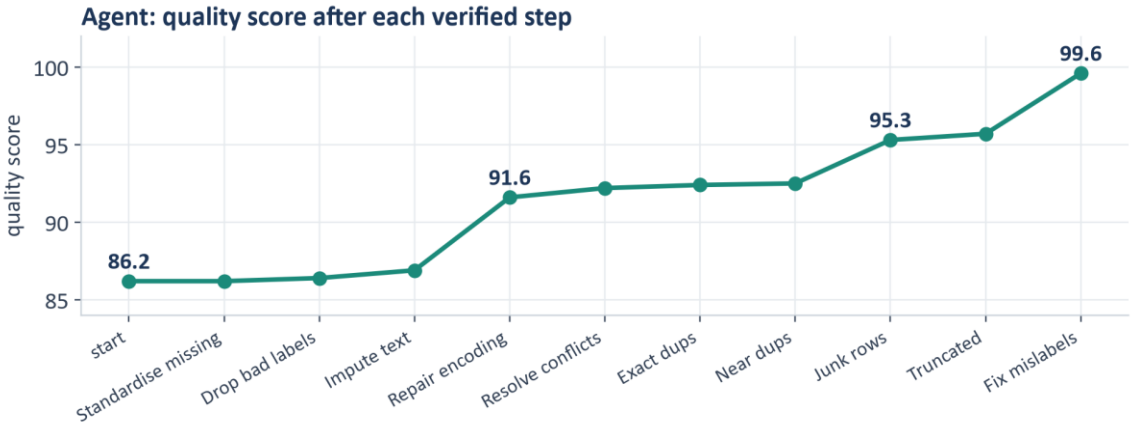


Figure 13: Quality score after each verified agent step on the benchmark

In 24 seconds the agent applied 10 steps and gave 2 pieces of advice (vocabulary size and sequence length, grouped splits for near duplicates). The score rose from 86.2 to 99.6 and it kept 6,785 rows where the fixed cleaner kept 6,428. It relabelled 285 of the 380 label flips back to their true class and deleted only 14% of them, and it lost fewer clean rows (261, 4.3%, against 330, 5.5%).

3.7 Does the cleaning help a model?

A quality score is only a proxy, so we trained the same model (TF-IDF + logistic regression) on four versions of the benchmark and evaluated each on the same 5,000 clean rows from the AG News training set that do not appear in the test set.

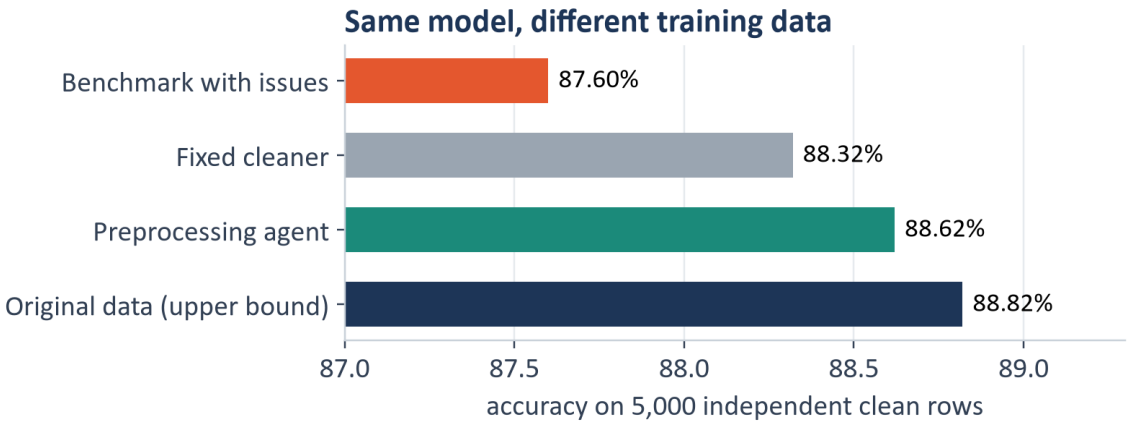


Figure 14: Accuracy of the same model trained on each version of the data

Trained on the dirty benchmark the model reaches 87.60%; after the fixed cleaner 88.32%; after the agent 88.62%; and on the original, uncorrupted test set 88.82%. The agent therefore recovers 84% of the accuracy lost to the injected problems ((88.62 − 87.60) / (88.82 − 87.60)) while training on fewer rows than the dirty data, so the removed and corrected rows were actively hurting the model.

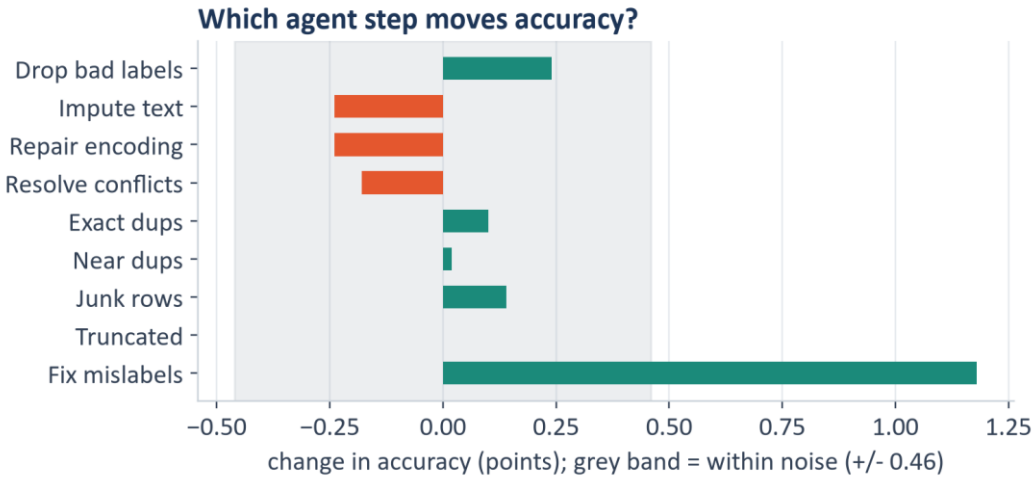


Figure 15: Change in accuracy after each agent step (one standard error is about 0.46 points)

With 5,000 evaluation rows one standard error is about 0.46 points, so most single steps are within noise. The one clearly significant step is fixing suspected mislabels (+1.18 points): wrong labels directly teach the model wrong answers. Duplicate removal mainly protects the evaluation, and encoding repair cleans the representation, rather than raising this model's accuracy.

3.8 Discussion

3.8.1 Questions the guideline asks us to answer

- **Basic data analysis: imbalance and outliers.** Classes are balanced, so no resampling was needed, but the recommender measures balance for any upload. Length outliers are real stories and were truncated at the 99th percentile rather than dropped. Duplicates,

conflicting labels and train/test leakage were removed, and broken encodings repaired (Sections 1.2.4 and 2.2).

- **How we evaluated the models.** The classifier on an untouched test set with accuracy, macro-F1, per-class precision and recall, a confusion matrix and calibration, against a TF-IDF benchmark. The detectors on a benchmark with known injected issues (recall, precision, ROC-AUC). The agent by retraining a model on its output and scoring it on independent clean data.
- **How the model changed with tuning.** A larger embedding and label smoothing helped and higher dropout hurt at short training length, but the eight configurations span only 0.7 points. The structural changes mattered more: masked max plus mean pooling, BatchNorm in the head, and for the autoencoder the switch from word to character features and the length calibration.
- **Is any model overfitted, and why?** The BiGRU mildly overfits after epoch 5 (train 98.3%, validation 92.3%) because a 6.3 M-parameter network can memorise 107,638 short texts. Regularisation keeps validation loss flat and early stopping keeps the best epoch. The autoencoder does not overfit (Section 3.2).

3.8.2 Did we have a benchmark model?

Yes. TF-IDF with logistic regression reached 92.32% test accuracy, 0.35 points above the BiGRU. We kept the BiGRU because the project needs calibrated probabilities for label auditing and a sequence model of the text, but on accuracy alone the benchmark wins. For the agent, the dirty data (87.60%) and the original uncorrupted data (88.82%) served as lower and upper benchmarks.

3.8.3 Did our plan of attack make sense?

Mostly yes. Building a benchmark with injected issues was the decision that made everything measurable, and it exposed a real flaw: the first autoencoder flagged short normal texts, which we would not have noticed by looking at the training loss. Measuring the agent by retraining a model showed which steps matter; a quality score alone would have suggested that every step helps equally. With hindsight, we spent too long on a from-scratch classifier that a linear baseline matches.

3.9 Hardware, memory and run time

Table 20: Hardware, memory and run time

Item	Value
GPU	NVIDIA GeForce RTX 3050 Laptop GPU, 4 GB, CUDA 12.8
CPU and RAM	Intel, 16 logical cores, 34 GB RAM (notebook used 3.6 GB)
Software	Python 3.14, PyTorch 2.11, scikit-learn, Streamlit
Hyperparameter search	183 s (8 configurations x 3 epochs, 60k rows)
Classifier training	142 s for 10 epochs, peak GPU memory 1.3 GB
Autoencoder training	47 s for 12 epochs, peak GPU memory 0.28 GB
Full audit of the 7.9k-row benchmark	13.8 s (GPU); 10 to 20 s on CPU in the application
Whole notebook	about 15 to 20 minutes

4.0 CONCLUSIONS

We built DQ-Recommender, which audits a text dataset automatically and turns the findings into severity-ranked issues, a 0 to 100 quality score and an ordered preprocessing plan, and a preprocessing agent that applies and verifies the fixes. The main results are:

- **Data.** AG News looks clean (no NaN, balanced classes) but a quarter of the rows have broken encodings, and there are conflicting duplicates and train/test leakage.
- **Models.** A BiGRU trained from scratch reaches 91.97% test accuracy (macro-F1 0.920) with a mild, contained overfit; a TF-IDF benchmark reaches 92.32%. A length-calibrated character autoencoder flags non-news text with 1.1% false alarms.
- **Detection.** On 14 injected issue types the system finds all rule-detectable issues, 91.3% of label flips (ROC-AUC 0.975) and all junk rows. On the real AG News test set it flags 261 rows (3.4%) as likely mislabelled.
- **Impact.** The agent lifts the quality score from 86.2 to 99.6 and recovers 84% of the accuracy lost to the injected problems. Fixing labels is what moved accuracy most.

4.1 The problem we solved

Data cleaning is normally a manual, one-off job: someone writes ad-hoc checks, eyeballs a sample and hopes nothing was missed. DQ-Recommender turns it into a repeatable, measured process. It finds problems that rules cannot see (wrong labels and junk text), explains each one in plain language with the code to fix it, applies the fixes in a safe order with a row-loss budget and automatic rollback, and proves the benefit on a model rather than on a score. A reviewer no longer reads the whole dataset: the label auditor ranks the 3 to 10% of rows worth checking, clearest cases first.

4.2 Use cases

Table 21 lists where the system is useful in practice and what each setting would need. The profiler, recommender and agent work on any labelled CSV; the label auditor and the autoencoder are trained on news and would be retrained on clean in-domain text for a new domain.

Table 21: Use cases of DQ-Recommender

Use case	Who uses it	How DQ-Recommender helps
Pre-training data gate	ML and MLOps teams	Audit every new labelled batch before it reaches training; block batches below a quality grade and attach the report and preprocess.py to the pipeline run.
News and content aggregation	Aggregators, media monitoring	Remove duplicated wire stories, strip scraping debris, and catch articles filed under the wrong category; this is the domain the models were trained on.
Annotation quality control	Labelling vendors, crowdsourcing teams	Rank suspected mislabels by margin so reviewers re-check a few percent of rows instead of all of them, and measure annotator error rates.
Support tickets and email routing	Customer service and IT teams	Find empty or placeholder fields, duplicate tickets and tickets routed to the wrong queue before a routing model is trained (classifier retrained on ticket categories).
Public and research datasets	Researchers, dataset curators	Publish a quality report with a benchmark dataset; on AG News it surfaced 261 likely mislabels and train/test leakage that are rarely mentioned.
Data migration and ETL checks	Data engineers	Catch disguised missing values ('N/A', '#NAME?'), broken encodings and truncated text after an export, import or system migration.

4.3 Limitations and future work

- **Domain.** The label auditor only knows the four AG News topics. Training a small model on each uploaded dataset would make label checks domain independent.
- **Accuracy and calibration.** Fine-tuning a pretrained encoder such as DistilBERT would add a few points of accuracy and better calibration, which directly improves the label auditor.
- **Auditing training data.** Out-of-fold probabilities would let the auditor check a training set without bias (the full confident-learning recipe [7]).
- **Splits and agent policy.** Use a grouped train/test split for near duplicates from day one, and let the agent choose between drop, relabel and impute by the measured downstream gain.

4.4 Lessons learned

Data that looks clean often is not; evaluate the detector, not only the model; and measure preprocessing by its effect on a model trained on the data, because a quality score alone suggests that every step helps equally.

5.0 REFERENCES

[1] Zhang, X., Zhao, J. and LeCun, Y. (2015). Character-level Convolutional Networks for Text Classification. *Advances in Neural Information Processing Systems 28* (NeurIPS 2015).

[2] amananandrai. AG News Classification Dataset. Kaggle. https://www.kaggle.com/datasets/amananandrai/ag-news-classification-dataset

[3] Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H. and Bengio, Y. (2014). Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. *Proceedings of EMNLP 2014*.

[4] Schuster, M. and Paliwal, K. K. (1997). Bidirectional Recurrent Neural Networks. *IEEE Transactions on Signal Processing*, 45(11), 2673-2681.

[5] Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J. and Wojna, Z. (2016). Rethinking the Inception Architecture for Computer Vision. *Proceedings of CVPR 2016*.

[6] Loshchilov, I. and Hutter, F. (2019). Decoupled Weight Decay Regularization. *ICLR 2019*.

[7] Northcutt, C., Jiang, L. and Chuang, I. (2021). Confident Learning: Estimating Uncertainty in Dataset Labels. *Journal of Artificial Intelligence Research*, 70, 1373-1411.

[8] Vincent, P., Larochelle, H., Bengio, Y. and Manzagol, P.-A. (2008). Extracting and Composing Robust Features with Denoising Autoencoders. *Proceedings of ICML 2008*.

[9] Guo, C., Pleiss, G., Sun, Y. and Weinberger, K. Q. (2017). On Calibration of Modern Neural Networks. *Proceedings of ICML 2017*.

[10] Paszke, A. et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems 32* (NeurIPS 2019).

[11] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

[12] Streamlit documentation. https://docs.streamlit.io